

ДОБАВЛЕНИЕ ИНТЕРАКТИВНОСТИ В СХЕМУ

Как статическая схема превращается в живую мнемосхему с контуром управления

1. От статической картинки к живому объекту

В документах «01-AI-schema-концепция» и «02-Описание работы системы AI-Schema» мы описали, как платформа превращает голосовые команды в структурированные SVG-схемы. Это мощный фундамент, но он описывает статический мир: объекты нарисованы, связи проведены, цвета заданы — и больше ничего не меняется.

Для целого ряда задач этого недостаточно:

- Мнемосхемы должны отражать текущее состояние оборудования (давление, температура, положение клапана).
- Приборные панели должны плавно менять показания стрелки или цифры. Панели оператора должны позволять управлять процессом: открывать задвижки, менять уставки, запускать насосы.
- Цифровые двойники должны реагировать на сигналы с IoT-датчиков в реальном времени.

Чтобы схема стала «живой», мы вводим новый уровень — интерактивный компонент. Ключевой принцип этого этапа: статика отвечает за форму, скрипт — за поведение.

2. Архитектура интерактивного компонента

Интерактивный компонент в Серверной Графической Модели теперь описывается тремя слоями:

2.1. View — «Тело» (SVG-разметка)

- Статическая визуальная часть: корпус прибора, рамка, базовые контуры, подписи.
- Генерируется LLM в виде SVG.
- Не меняется в процессе работы.
- Содержит элементы с `id`, к которым будет обращаться скрипт (например, `id="needle"`, `id="lamp"`).

2.2. State — «Память» (Состояние)

Переменные, хранящие текущие значения компонента:

- `value` — текущее показание,
- `min_val`, `max_val` — диапазон,
- `status` — состояние («open» / «closed»),
- `threshold` — пороговые значения.

Это и есть Single Source of Truth для данного компонента.

2.3. Logic — «Нервная система» (JavaScript)

Код, сгенерированный LLM, который связывает изменение State с изменением View и обрабатывает события пользователя.

JavaScript — это полноправная часть DOM и веб-стандарта. Он генерируется языковой моделью и передаётся в браузер тем же способом, что и SVG-разметка.

3. Две ключевые роли скрипта

3.1. Инициализация: программное построение повторяющихся элементов

Некоторые элементы крайне неудобно описывать как набор статических SVG-тегов. Например, шкала вольтметра с 50 делениями — это 50 отдельных линий, каждая со своими координатами и подписями. Генерировать их поштучно через LLM — неэффективно и избыточно. Вместо этого скрипт при загрузке компонента в цикле размечает все деления, подписи и цветовые зоны (норма / предупреждение / авария) по заданным параметрам:

Пример кода:

```
function init()
{
    for (let v = min_val; v <= max_val; v += step)
    {
        let angle = start_angle + (v - min_val) / (max_val - min_val) * (end_angle - start_angle); drawTick(angle, v);
    }
}
```

Результат тот же — точная шкала на экране, но создана она программно за миллисекунды, без раздувания JSON-модели.

Аналогия: вместо того чтобы вручную чертить 50 чёрточек на линейке, вы отдаёте одну команду «разметь от 0 до 300 с шагом 6».

3.2. Обработчики событий: реакция на изменения

Скрипт содержит функции, которые срабатывают в ответ на:

- Внешние данные (сигнал с датчика) → поворачивается стрелка, обновляется цифра.
- Действия пользователя (клик, перетаскивание) → открывается клапан, переключается тумблер.
- Пороговые условия (значение превысило норму) → загорается аварийная лампа.

4. Примеры интерактивных компонентов

4.1. Аналоговый прибор (вольтметр, манометр)

- View: корпус, шкала (статика), стрелка с transform-origin в центре.
- State: value, min_val, max_val, start_angle, end_angle.
- Logic: при изменении value пересчитывает угол и применяет transform: rotate() к стрелке. При превышении аварийного порога — включает красную зону.

4.2. Цифровой 7-сегментный индикатор

- View: 7 SVG-сегментов (A–G) + десятичная точка (DP).
- State: digit, show_dot.
- Logic: содержит таблицу истинности (какие сегменты зажигать для каждой цифры). При изменении digit гасит все сегменты и зажигает нужные.

4.3. Управляющий элемент (клапан на трубе)

- View: труба, корпус клапана, рукоятка.
- State: status, flow_rate. • Logic: при status = "open" поворачивает рукоятку, окрашивает трубу в цвет потока, запускает анимацию движения. При status = "closed" — всё возвращает в исходное состояние.
- Событие onClick: переключает состояние и отправляет команду на сервер.

4.4. Вращающийся ротор (насос, двигатель)

- View: корпус, группа элементов ротора.
- State: rpm, is_running.
- Logic: длительность CSS-анимации обратно пропорциональна rpm. При остановке — анимация выключается.

Синхронизация состояний: как это работает

Чтобы все участники совещания видели одинаковую картину, используется механизм Single Source of Truth:

1. Локальный отклик (UI). Когда приходит событие, JS в браузере мгновенно меняет DOM — стрелка поворачивается без задержек.
2. Отправка на сервер от источника событий. Браузер передаёт событие по WebSocket.
3. Обновление Истины. Сервер обновляет State в Серверной Графической Модели.
4. Broadcast. Сервер рассылает «дельту» всем другим подключённым браузерам — у них стрелки поворачиваются синхронно.

6. Контур управления: кнопки и поля ввода

Интерактивная мнемосхема — это не только мониторинг, но и управление. На панель добавляются:

- Поля ввода (input, select) — для изменения уставок, параметров, целевых значений.
- Кнопки действий — для запуска/остановки, открытия/закрытия, подтверждения.

Обязательное правило безопасности: любое управляющее действие вызывает детерминированное модальное окно подтверждения с явным указанием:

- что меняется,
- старое и новое значение,
- кто выполняет действие,
- что действие будет запротоколировано.

Это исключает случайные клики и обеспечивает аудируемость.

7. Голосовое управление: возможности и риски

Логичный вопрос: если схему можно создавать голосом, можно ли и управлять ей голосом? «Открой клапан К-1», «Установи давление на 50 бар».

Технически это возможно. Но здесь кроется критически важный риск, который необходимо осознать до внедрения.

7.1. Почему опасно использовать LLM в рабочем контуре управления

В документах 01 и 02 мы зафиксировали принцип: LLM работает только на этапе конструирования. Для контура управления этот принцип становится вопросом промышленной безопасности. Использование большой генеративной языковой модели (GigaChat, Qwen, GPT) для исполнения управляющих команд несёт в себе четыре неприемлемых риска:

Риск 1. Галлюцинации и искажение параметров

Генеративная модель по своей природе — вероятностная. Она может:

- «услышать» команду там, где её не было,
- исказить числовой параметр («двести пятьдесят» превратить в «двести пятнадцать»),
- додумать объект, которого нет на схеме,
- выполнить команду в неправильном контексте.

В конструировании схемы это приводит к неточному чертежу. В управлении реальным объектом — к аварии, остановке производства, финансовым потерям или угрозе жизни.

Риск 2. Утечка конфиденциальных данных

Голосовые команды содержат названия узлов, параметры, уставки, имена операторов. Передача этих данных в облачную LLM означает выход за периметр безопасности предприятия. Для режимных объектов (энергетика, ВПК, нефтегаз) это недопустимо.

Риск 3. Недетерминированность результата

Одна и та же команда, поданная дважды, может быть интерпретирована большой LLM по-разному. В контуре управления это неприемлемо: система должна вести себя предсказуемо при одинаковых входных данных.

Риск 4. Задержки и зависимость от сети

Ответ большой LLM занимает от 1 до 5 секунд. Для оперативного управления (аварийная остановка, регулировка) это критично. Кроме того, зависимость от внешнего сервиса создаёт точку отказа.

Вывод: использовать LLM общего назначения в рабочем контуре управления — опасно и архитектурно неверно. Это нарушает фундаментальный принцип «AI для конструирования, детерминированный код для работы».

7.2. Гибридная архитектура: правильное разделение контуров

Чтобы сохранить философию проекта, система разделяется на два независимых контура:

Контур 1: Конструирование

Назначение: Создание и редактирование схемы

Чем обрабатывается: Большая LLM (облачная или локальная)

Требования: Творчество, понимание контекста, задержка 2–5 с допустима

Контур 2: Исполнение и управление

Назначение: Реагирование на события, управление объектами

Чем обрабатывается:

Детерминированный JS + узкая NLU-модель

Требования: Скорость < 200 мс, нулевые галлюцинации, работа в закрытом контуре

7.3. Узкая специализированная модель — правильное решение для голоса в управлении

Для голосового управления нам не нужна генеративная модель. Нам нужна модель распознавания намерений (***Intent Classification***) и извлечения сущностей (***Named Entity Recognition***).

Это легковесная локальная модель (дообученный BERT, Rasa, или собственный классификатор), которая знает строго ограниченный белый список шаблонов:

- [ACTION: OPEN, TARGET: VALVE_1]
- [ACTION: SET_PARAM, TARGET: TEMP, VALUE: 250]
- [ACTION: STOP, TARGET: PUMP_3]

Преимущества такого подхода:

- Скорость реакции — менее 200 мс.
- Нулевые галлюцинации: если фраза не попадает в белый список, система отвечает «Команда не распознана», а не придумывает действие.
- Работа на слабом локальном железе.
- Полная изоляция данных — ничего не покидает периметр.

7.4. Защищённый пайплайн голосовой команды

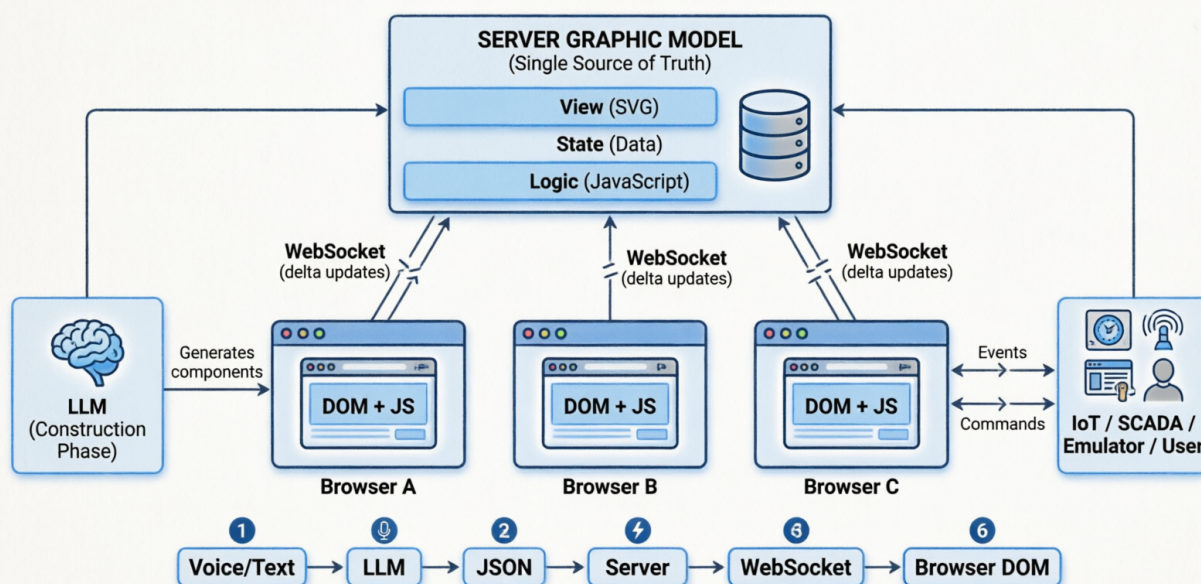
Если голосовое управление всё же включено, команда проходит через строгий детерминированный пайплайн:

- Голос → Текст (Local STT). Локальное распознавание речи (Whisper.cpp, Vosk). Аудио не уходит в облако.
- Текст → Намерение (Narrow NLU). Легковесная модель проверяет, попадает ли фраза в белый список шаблонов.
- Валидация параметров (Детерминированный код). JS на стороне браузера проверяет допустимость значений (температура не может быть 9999°C, давление — в пределах рабочего диапазона).
- Запрос подтверждения (UI). Система визуально и голосом переспрашивает: «Принята команда: установить давление на 50 бар. Подтвердите».
- Отправка на сервер. Только после явного подтверждения (голосом «Да» или кликом) команда уходит по WebSocket, логируется (кто, когда, что) и передаётся в физическую систему.

8. Итоговая схема взаимодействия интерактивного компонента

- События от внешних систем и пользователей приходят на сервер.
- Сервер обновляет State и рассылает дельты во все браузеры.
- JS в каждом браузере детерминированно обновляет View.
- Управляющие команды от пользователя проходят через валидацию и подтверждение.

AI-Schema Platform: Interactive Component System Architecture



9. Ключевые преимущества для бизнеса

1. Мнемосхемы оживают. Объекты на схеме начинают отражать реальное состояние оборудования в реальном времени.
2. Скорость создания. LLM генерирует и статику, и скрипт по одному голосовому описанию. Инженеру не нужно программировать.
3. Безопасность и предсказуемость. Скрипт генерируется один раз при создании компонента и далее работает детерминированно — никаких «галлюцинаций» в рантайме.
4. Масштабируемость через библиотеки. Один раз созданный интерактивный компонент (вольтметр, клапан, индикатор) попадает в отраслевую библиотеку и переиспользуется в других проектах.
5. Защищённое управление. Разделение контуров конструирования и исполнения гарантирует, что критические команды не могут быть искажены AI.
6. Импортозамещение. Платформа становится отечественной альтернативой западным SCADA/CAD-системам с современным AI-интерфейсом.

10. Заключение

Добавление интерактивности — это естественная эволюция AI-Schema. Мы переходим от «рисования картинок» к семантическому конструированию живых моделей.

При этом мы сохраняем архитектурную дисциплину:

- AI (LLM) остаётся инструментом конструирования — там, где нужны творчество и понимание контекста.
- Детерминированный код (JS + узкая NLU) берёт на себя исполнение и управление — там, где нужны скорость, предсказуемость и безопасность.
- Именно это разделение позволяет превратить AI-Schema из удобного редактора схем в полноценную платформу для цифровых двойников, мнемосхем и панелей оператора — без компромиссов в надёжности.

AI-Schema превращает совещания из обсуждений в акты созидания — а интерактивные схемы превращают эти акты в управляющие воздействия на реальный мир.